

Training the model to know Indian groceries, not just "objects"

You're right that this is the hard part — and the part a generic vision model can't do out of the box. Here's how to actually build it: what to collect, how to train it so new SKUs never need retraining, and a realistic order to tackle rice, dal grades, masalas, packaged goods, and produce.

00 Two different things	01 Difficulty tiers	02 Collecting data
03 Don't start from zero	04 Training method	05 The dal-grade problem
06 Where training runs	07 Evaluation targets	08 The improvement loop
09 Realistic scope		

00 Onboarding a SKU vs. training the model — not the same step

Worth separating these clearly, because the plan changes depending on which one you mean:

- **Onboarding** (4 photos, 30 seconds) — adds a new reference fingerprint to the catalog, using a model that is *already trained*. This is cheap and is what happens every time a store stocks a new item.
- **Training** (what this page is about) — teaches the underlying model how to tell Indian groceries apart *in the first place*: that Basmati and Sona Masoori are different, that broken-grain toor dal looks different from whole-grain, that a green chili and a cucumber are not the same thing at a glance. You do this once (and periodically improve it), on a proper dataset — not per SKU.

Get training right and onboarding stays cheap forever. Get it wrong and every new SKU becomes a retraining project — exactly what your "no retraining" pitch is trying to avoid.

01 Not all products are equally hard — tier them

"All Indian products" is really three very different computer-vision problems wearing one trenchcoat. Treat them separately and the hard 20% stops eating budget meant for the easy 80%.

Difficulty tiers and what each one actually needs

TIER	EXAMPLES	WHAT MAKES IT HARD	WHAT IT NEEDS
TIER 1	Branded packets — biscuits, snacks, spice pouches, bottled goods	Barely hard — distinct box art, logos, colors	Standard classification; barcode/weight can do most of the confirming
TIER 2	Rice varieties, dal types & quality grades, whole/powdered masalas	Visually near-identical between variants; grades differ subtly	Fine-grained fingerprint model + weight/density fusion — your actual moat
TIER 3	Fruits & vegetables	No fixed shape — size, ripeness and color shift daily	Category-level recognition (not exact variety) + weight-driven pricing

WHY THIS MATTERS

Tier 1 is where a lot of your SKU count lives but almost none of your risk. Tier 2 is where the device either earns its "unique" claim or doesn't. Tier 3 is a different problem again — don't try to solve it with the same approach as Tier 2.

02 Collecting the training data, tier by tier

TIER 1

Packaged goods

☐ Reuse the 4-photo onboarding flow

it's already enough signal — distinct packaging does most of the work.

☐ Scale it by crowdsourcing across branches

whoever restocks a new packaged item photographs it on the spot; no dedicated data-collection trip needed.

TIER 2 Rice, dal, masalas — the fine-grained set

- ❑ **Build one fixed capture rig**
same lighting, same camera angle and distance you'll use in the actual kiosk tray — consistency here matters more than volume.
- ❑ **Source real graded samples, not just "a bag of dal"**
get physical samples of each quality grade from your supplier or a wholesale mandi contact — you cannot photograph a quality difference you don't have in hand.
- ❑ **Shoot 100-300 images per variety/grade**
vary fill level (empty tray → full), natural lighting drift across the day, and loose-pile arrangement.
- ❑ **Log the density number alongside every shot**
 $\text{weight} \div \text{estimated tray-fill volume}$ — this is training data for the numeric feature, not just the image.

TIER 3 Fruits & vegetables

- ❑ **Capture across a freshness timeline**
the same batch photographed on day 1, 3 and 5 — produce genuinely looks different as it ages, and the model needs to see that range.
- ❑ **Shoot multiple specimens, not one "reference" item**
30-50 images per type across natural size and ripeness variation — a single perfect photo of one tomato won't generalize.

03 Don't start from zero

Two different kinds of head start, for two different tiers:

- **The backbone itself:** start from an ImageNet-pretrained MobileNetV4 (or similar), not a randomly initialized network. You're fine-tuning an existing visual understanding, not

teaching a model to see from scratch — that's what makes a few hundred images per category realistic instead of needing millions.

- **Produce specifically:** public datasets exist (Fruits-360, VegFru, and similar) and can pretrain the produce-recognition head before you fine-tune on your own photos. They skew Western, though — okra, drumstick, bitter melon and other India-specific produce will need your own collection regardless.
 - **Rice/dal/masala varieties and quality grades:** there isn't a strong public dataset for this — which is exactly why it's a real moat. No shortcut here; this is the collection work in Section 02 that only you're doing.
-

04 Train for "how similar," not "which class"

The training method is what actually delivers your "no retraining, 30-second onboarding" promise — and it's not a standard image classifier. A classifier bakes in a fixed list of SKUs; add one and you have to retrain. Instead, fine-tune the embedding model with **metric learning** (triplet or contrastive loss): it learns a general sense of "how visually similar are these two items," and a new SKU is just a new point in that space — no retraining, ever.

□ Build training triplets

anchor photo, a positive (same SKU/variety), and a negative (a different one).

□ Make the negatives hard on purpose

explicitly pair Basmati against Sona Masoori, whole toor dal against broken toor dal — the confusions you actually care about, not random unrelated items.

□ Fine-tune the backbone against this triplet objective

on the labeled Tier 2/3 dataset from Section 02.

□ Re-populate the FAISS reference index from the new embeddings

this is the only "onboarding-side" step that changes — the model itself is done.

05 The honest hard part: dal quality grades

Telling toor dal from moong dal is a variety problem — hard, but solvable with the approach above. Telling *premium* toor dal from *standard* toor dal is a grading problem, and

it's genuinely harder: the visual differences (broken-grain %, color uniformity, dust) are subtler than variety differences, and a wide top-down tray shot may not resolve them well.

RECOMMENDATION

Don't ask the camera to certify grain quality from a checkout-distance photo alone. Instead, tie the quality grade to *which bin or hopper* the store loaded — when staff restock, they declare "this hopper is Grade A toor dal" once; vision only needs to confirm "this is toor dal" (a variety problem it's good at), and the system trusts the declared grade for whatever's currently in that hopper. Treat pure vision-based grading as a stretch goal to chase after launch, not a launch requirement.

06 Where the training actually runs

Now — Mac M4 via PyTorch + MPS

your datasets are hundreds to low-thousands of images per category; M4 unified memory handles this fine, no cloud GPU needed yet.

Later — a rented cloud GPU (A10/A100-class)

once real pilot data pushes your dataset into the tens of thousands of images, for faster iteration. Not needed for the initial build.

Always — re-validate accuracy after RKNN INT8 quantization

a model that's 96% accurate in PyTorch on the M4 is not guaranteed to still be 96% accurate after quantization on the real RK3588 board. Check both, every time, per the Track B gate in the build roadmap.

07 What "good enough" looks like, per tier

Target accuracy before calling a tier launch-ready

TIER	TARGET	FALLBACK FOR THE REST
TIER 1Packaged	≥99% top-1	Should rarely be needed — packaging is distinctive
TIER 2Variety (rice/dal/masala)	≥95% top-1	Show top-2 candidates on the touchscreen for a 1-tap confirm
TIER 2Quality grade	≥85% (or hopper-declared)	Defer to declared hopper grade, per Section 05

WHY A TOUCHSCREEN FALLBACK, NOT 100% ACCURACY

Forcing every tier to near-100% before launch is how this project stalls. A low-confidence item showing its top two guesses for a one-tap confirm keeps the customer moving in under a second — genuinely faster than chasing the last few points of model accuracy, and it's what grab-and-go systems elsewhere do for exactly this reason.

08 The loop that gets you the rest of the way

You will not pre-collect every Indian grocery item before launch, and you don't need to — the model keeps learning from real store use if you wire the feedback path in from day one.

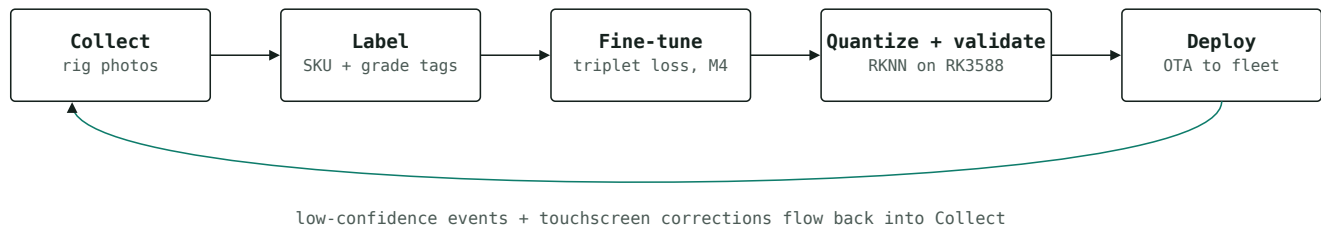


Fig 1 — every low-confidence or corrected checkout becomes tomorrow's training data, closing the loop without a dedicated re-collection trip.

Practically: tag every transaction as auto-confirmed or customer-corrected in the same event log your analytics already use. A corrected item is a free, real-world labeled example — periodically (monthly is reasonable to start) pull those into the next fine-tuning run.

09 A realistic order to actually do this in

"All Indian products" as a day-one training target would stall the pilot for months. Scope it in waves instead — the loop in Section 08 is what gets you to full coverage, not a single

upfront data-collection marathon.

Suggested phasing

PHASE	COVERS	WHEN
1	Top 50-80 SKUs by expected sales volume, across all three tiers — enough for a believable pilot	Alongside Track A/B of the build roadmap
2	Full rice/dal/masala variety + grade set you actually stock	Before scaling past the first pilot store
3	Everything else, via the feedback loop above	Continuous, post-launch

Pick the Phase 1 list with whoever owns merchandising — the 50-80 items customers actually buy most, deliberately including a couple of your hardest lookalike pairs (a rice variety pair, a dal grade pair) so the pilot proves the hard case, not just the easy 80%.